

Informe de Resultados — Análisis Estático, Métricas y Refactorización

Proyecto	FuBanking · Módulo Bolsillos (Pockets)
Herramienta	SonarQube Community Build (self-hosted, Docker) · project key: FuBank
Fecha	2026-09-14
Complementa	Plan de Pruebas — Módulo Bolsillos

0. Estado de la rúbrica

#	Criterio	Estado
1	Presentación + plan de pruebas	Completo
2	Pruebas unitarias (AAA + FIRST + 5 dobles)	Completo — 58 pruebas, 100% cobertura
3	SonarQube (1.ª ejecución)	Capturado (este informe)
4	Métricas manuales	En Metricas-Bolsillos.xlsx
5	Refactorización + reejecución	Refactor hecho; falta reejecutar Sonar
6	Comparación manual vs Sonar	1.ª ejecución hecha; falta la 2.ª
7	Quality Gate estricto	Gate creado; falta que el proyecto lo pase

3. Resultados de SonarQube — 1.ª ejecución (línea base)

Análisis del proyecto completo (todos los módulos del equipo): 15.050 líneas de código.

3.1 Instantánea global

Indicador	Valor	Nota
Quality Gate	Passed	Gate 'Sonar way' (evalúa código nuevo)
Bugs	6	Reliability rating C
Vulnerabilidades	0	Security rating A
Code Smells	156	Maintainability rating A
Deuda técnica	903 min (~15 h)	sqale_index (proyecto completo)
Cobertura	96.2%	línea 96.4% · rama 95.8%
Duplicación	2.5%	469 líneas duplicadas
Security Hotspots	14	Revisados 0% → rating E

Evolución: 1.ª corrida (13-sep) 167 issues / 42% cobertura / 2.8% duplicación → actual 96.2% cobertura / 2.5% duplicación.

3.2 Cobertura del módulo Bolsillos (backend)

Archivo	Cobertura	Cobertura de rama
CreatePocket.ts	96.3%	—
UpdatePocket.ts	92.0%	88.2%
DeletePocket.ts	89.8%	—
TransferPocketBalance.ts	91.3%	85.2%
GetAccountPockets.ts	92.1%	—
Pocket.ts (entidad)	97.8%	92.3%

Esta cobertura proviene de la ronda manual (c8). Tras integrar las pruebas unitarias Vitest (AAA + 5 dobles), el módulo llega a 100% en líneas y ramas (ver §5).

4. Métricas manuales (resumen)

Calculadas estáticamente sobre el código (detalle en Metricas-Bolsillos.xlsx). Convención: $V(G) = \text{Decisiones} + 1$ a nivel de predicado.

Caso de uso (backend)	V(G)	Cognitiva	SLOC	CBO	DIT
CreatePocket	6	5	52	6	1
GetAccountPockets	2	1	19	3	1
UpdatePocket	11	16	65	6	1
DeletePocket	5	4	62	6	1
TransferPocketBalance	10	9	63	6	1

- SLOC del módulo backend: 373 (combinado BE+FE: 544).
- Defectos documentados ($E = 5$): D-01...D-05 (4 en Crear, 1 en Actualizar).
- Acoplamiento (CBO): 6 en la mayoría (3 interfaces de repositorio + entidades). Herencia (DIT): 1.
- Duplicación estimada: ~8% (bloque de notificación repetido en los 5 casos de uso).

6. Comparación: métricas manuales vs SonarQube

6.1 Complejidad y tamaño

Caso de uso	V(G) manual	Ciclo. Sonar	Cognit. manual	Cognit. Sonar	SLOC	ncloc
CreatePocket	6	7	5	5	52	52
GetAccountPockets	2	4	1	1	19	19
UpdatePocket	11	17	16	16	65	65
DeletePocket	5	6	4	4	62	62
TransferPocketBalance	10	11	9	9	63	63

6.2 Interpretación de las diferencias

- Complejidad cognitiva: coincidencia EXACTA (5,1,16,4,9) → valida el cálculo manual (misma definición de SonarSource).
- Líneas de código: coincidencia EXACTA (SLOC manual = ncloc Sonar).
- Complejidad ciclomática: Sonar reporta más (+1 a +6). Son dos convenciones: manual $V(G)=D+1$ a nivel de predicado; Sonar suma +1 por cada operador lógico (&&, ||). La mayor brecha está en UpdatePocket (11 vs 17), el método con más condiciones compuestas.
- Conclusión: las tres métricas concuerdan en el diagnóstico — UpdatePocket es el caso de uso más complejo y el candidato a refactor, señalado por igual manualmente y por Sonar.

6.3 Coincidencias de calidad

- Vulnerabilidades: manual 0 = Sonar 0 (Security A).
- Cobertura: el análisis manual anticipaba brechas en Eliminar y Transferir; Sonar lo confirma (DeletePocket 89.8%, rama de TransferPocketBalance 85.2%).

5. Refactorización

Unidad refactorizada: UpdatePocket.execute — la de mayor complejidad según ambos análisis.

Técnica: Extract Method — se dividió el método monolítico en cuatro métodos privados con responsabilidad única (validateInput, loadAuthorized, adjustAmount, notify) y se adelantó la validación de 'sin cambios' (falla rápido antes de tocar la base de datos).

Métrica (máximo por función)	Antes	Después
Complejidad cognitiva	16 (crítica)	6
Complejidad ciclomática	17	7
Nº de funciones	1	5
Code smell 'Cognitive Complexity too high'	Presente	Eliminado

Verificación: las 58 pruebas unitarias siguen en verde y la cobertura del módulo se mantiene en 100% (líneas y ramas) — el comportamiento observable es idéntico.

Pendiente: reejecutar SonarQube con la rama integrada para reflejar la cobertura de Bolsillos al 100% y la caída de complejidad/deuda de UpdatePocket. Los valores 'después' son los esperados; la 2.ª instantánea los confirmará.

7. Quality Gate estricto

Se creó el gate 'Quality Gate Gabriel' con condiciones sobre código global:

Condición	Umbral (falla si...)	Estado global
Issues	> 0	hay issues abiertos
Security Hotspots Reviewed	< 100%	0% revisados
Coverage	< 90%	96.2%
Duplicated Lines (%)	> 2%	2.5%
Maintainability Rating	peor que A	A
Reliability Rating	peor que A	C
Security Rating	peor que A	A
Technical Debt Ratio	> 90%	ratio bajo

Observaciones importantes:

1. La rúbrica pide 'deuda técnica ≤ 90 MINUTOS', no 'ratio de deuda ≤ 90%'. La condición actual (Technical Debt Ratio) casi nunca falla. Para cumplir la rúbrica literalmente hay que usar la métrica Technical Debt (minutos); sobre el proyecto completo son 903 min, así que solo es alcanzable sobre código nuevo (new_technical_debt = 0).

2. Las condiciones se evalúan sobre TODO el proyecto, no solo Bolsillos. Que el gate pase es trabajo de equipo: corregir los 6 bugs (Reliability → A), revisar los 14 Security Hotspots y bajar la duplicación de 2.5% a $\leq 2\%$.
3. El módulo Bolsillos por sí solo ya está limpio: 0 bugs, cobertura 100%, UpdatePocket refactorizado y sin el smell de complejidad.

Recomendación: aplicar el gate estricto en modo Clean as You Code (condiciones sobre New Code), donde FuBank ya cumple: cobertura nueva 100%, duplicación nueva 0%, deuda nueva 0, ratings A. Así el criterio 7 se satisface de forma sostenible.

8. Pendientes para cerrar la entrega

4. Subir la rama tests/bolsillos-unitarias-aaa (pruebas + refactor) al repo que analiza SonarQube.
5. Reejecutar el scanner → 2.ª instantánea ('después'): Bolsillos a 100% y UpdatePocket con complejidad/deuda reducidas.
6. Completar §5 y §6 con los números 'después' reales de Sonar.
7. Corregir la condición de deuda del gate (minutos, no ratio) y decidir alcance (proyecto vs código nuevo).
8. (Equipo) Corregir bugs de Reliability y revisar Security Hotspots para que el gate estricto pase en verde.